LAB SHEET 4

Tasks:

Task 1: Custom Modules :

Create sum.js and even-odd.js exporting a function each, then import both into app1.js and call them with sample values.

Files: sum.js, even-odd.js, app1.js Run: node app1.js

Solution -> Sum.js

```
Lab Sheet 4 > JS Sum.js > ...
1  function sum(a, b) {
2      return a + b;
3  }
4  module.exports = sum;
```

even-odd.js

```
Lab Sheet 4 > JS even-odd.js > ...
1  function evenOdd(num) {
2      if (num % 2 === 0) {
3          return "Even";
4      } else {
5          return "Odd";
6      }
7  }
8  module.exports = evenOdd;
```

app1.js

```
Lab Sheet 4 > JS app1.js > ...
1  const sum = require("./Sum");
2  const evenOdd = require("./even-odd");
3  console.log("Sum:", sum(10, 20));
4  console.log("10 is:", evenOdd(10));
5  console.log("7 is:", evenOdd(7));
```

Output ->

```
Problems  Output  Debug Console  Terminal  Ports  Query Results        powershell - Lab Sheet 4
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode> cd 'c:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4'
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4> node .\app1.js
Sum: 30
10 is: Even
7 is: Odd
```

Task 2: File System — CRUD on a Text File

Write file-ops.js that: creates student.txt, writes your name and roll number to it, appends a course name, reads and prints the full content, then renames the file to profile.txt.

File: file-ops.js Use: fs.writeFile, fs.appendFile, fs.readFile, fs.rename

Solution -> file-ops.js

```
Lab Sheet 4 > JS file-ops.js > ...
1  const fs = require("fs");
2  fs.writeFile("student.txt", "Name: Vikrant Tyagi\nRoll Number: 75", (err) => {
3      if (err) throw err;
4      console.log("student.txt created and data written.");
5      fs.appendFile("student.txt", "\nCourse: Full Stack Web Development", (err) => {
6          if (err) throw err;
7          console.log("Course appended successfully.");
8          fs.readFile("student.txt", "utf8", (err, data) => {
9              if (err) throw err;
10             console.log("\nFile Content:");
11             console.log(data);
12             fs.rename("student.txt", "profile.txt", (err) => {
13                 if (err) throw err;
14                 console.log("\nFile renamed to profile.txt");
15             });
16         });
17     });
18 });
```

profile.txt

```
Lab Sheet 4 > profile.txt
1  Name: Vikrant Tyagi
2  Roll Number: 75
3  Course: Full Stack Web Development
4
```

Output ->

```
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4> node .\file-ops.js

File Content:
Name: Vikrant Tyagi
Roll Number: 75
Course: Full Stack Web Development

File renamed to profile.txt
```

Task 3: Directory Operations

Write dir-ops.js that creates a folder named uploads, creates 3 empty files inside it, lists all files in the folder, then deletes one of the files.

File: dir-ops.js Use: fs.mkdir, fs.writeFile, fs.readdir, fs.unlink

Solution -> dir-ops.js

```
Lab Sheet 4 > JS dir-ops.js > ...
1  const fs = require("fs");
2  fs.mkdir("uploads", { recursive: true }, (err) => {
3      if (err) throw err;
4      console.log("uploads folder created.");
5      fs.writeFile("uploads/file1.txt", "", (err) => {
6          if (err) throw err;
7          fs.writeFile("uploads/file2.txt", "", (err) => {
8              if (err) throw err;
9              fs.writeFile("uploads/file3.txt", "", (err) => {
10                 if (err) throw err;
11                 console.log("3 files created.");
12                 fs.readdir("uploads", (err, files) => {
13                     if (err) throw err;
14                     console.log("\nFiles in uploads:");
15                     console.log(files);
16                     fs.unlink("uploads/file3.txt", (err) => {
17                         if (err) throw err;
18                         console.log("\nfile3.txt deleted.");
19                     });
20                 });
21             });
22         });
23     });
24 });
```

Output ->

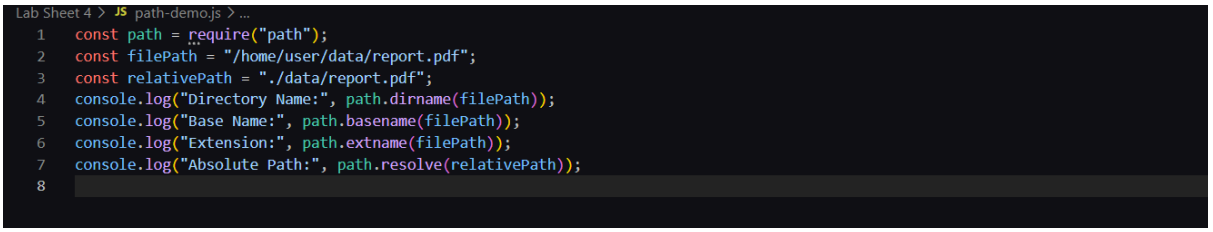


Task 4: Path Module

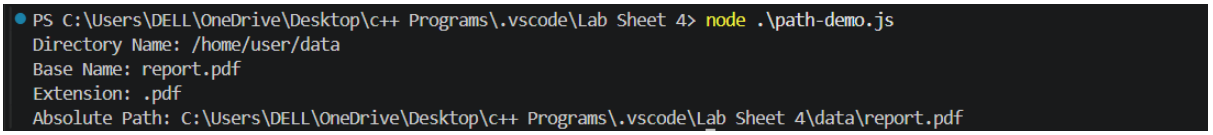
Write path-demo.js that takes a hardcoded file path (e.g. /home/user/data/report.pdf) and prints its directory name, base name, extension, and an absolute path resolved from a relative one.

File: path-demo.js Use: path.dirname, path.basename, path.extname, path.resolve

Solution -> path-demo-js



Output ->

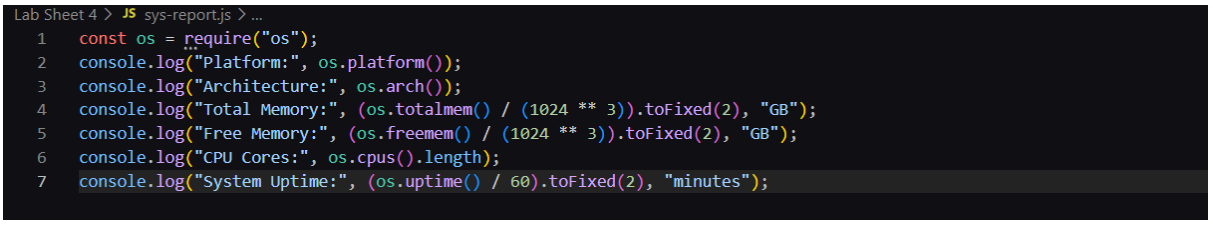


Task 5: OS Module Report

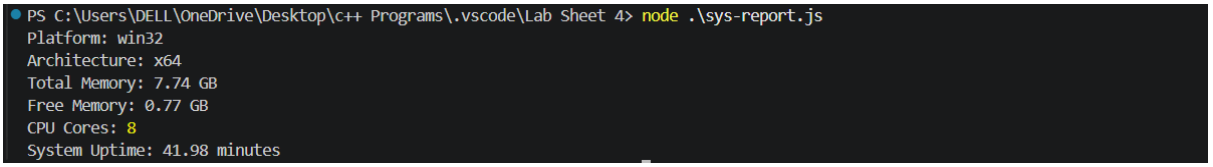
Write sys-report.js that prints platform, architecture, total and free memory, number of CPU cores, and system uptime in minutes.

File: sys-report.js

Solution -> sys-report.js



Output ->



Task 6: Events Module — Custom EventEmitter

Write order-events.js that defines an EventEmitter for an 'orderPlaced' event. On emit, pass an order object (id, item, amount) and log a formatted receipt. Emit the event 3 times with different orders.
File: order-events.js Use: require('events').EventEmitter

Solution -> order-events.js

```
Lab Sheet 4 > JS order-events.js > ...
1   const EventEmitter = require("events").EventEmitter;
2   const orderEmitter = new EventEmitter();
3   orderEmitter.on("orderPlaced", (order) => {
4       console.log("----- Order Receipt -----");
5       console.log("Order ID:", order.id);
6       console.log("Item:", order.item);
7       console.log("Amount: ₹" + order.amount);
8       console.log("-----------------------\n");
9   });
10  orderEmitter.emit("orderPlaced", {
11      id: 101,
12      item: "Laptop",
13      amount: 55000
14  });
15  orderEmitter.emit("orderPlaced", {
16      id: 102,
17      item: "Keyboard",
18      amount: 1500
19  });
20  orderEmitter.emit("orderPlaced", {
21      id: 103,
22      item: "Mouse",
23      amount: 800
24  });
```

Output ->

```
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4> node .\order-events.js
----- Order Receipt -----
Order ID: 101
Item: Laptop
Amount: ₹55000
-----------------------

----- Order Receipt -----
Order ID: 102
Item: Keyboard
Amount: ₹1500
-----------------------

----- Order Receipt -----
Order ID: 103
Item: Mouse
Amount: ₹800
```

Task 7: HTTP Server with Manual Routing

Write server.js (no framework) with routes: GET / returns a welcome message, GET /students returns a hardcoded JSON array of 3 students, GET /students/:id (parse the id from req.url) returns the matching student or a 404 JSON error, and any other route returns a 404 page.
File: server.js Run: node server.js → test all 4 routes in the browser or Postman

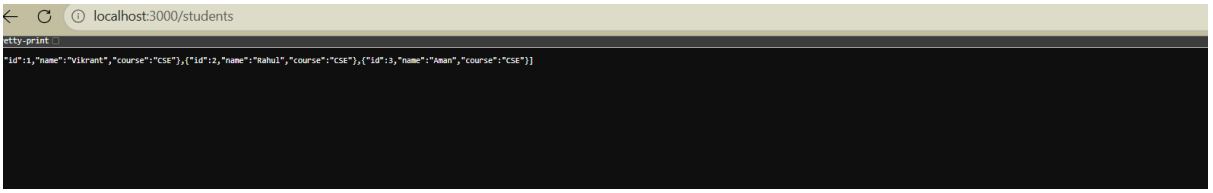
Solution -> server.js

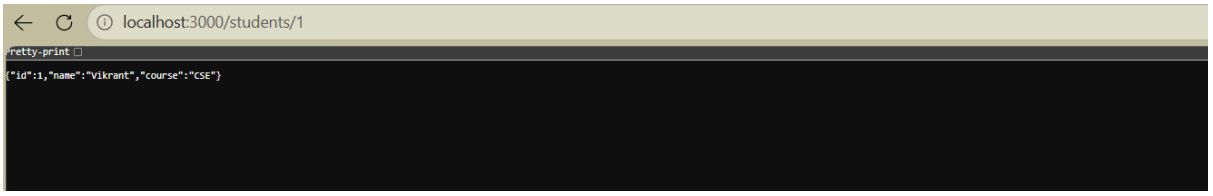
Lab Sheet 4 > JS server.js > ...

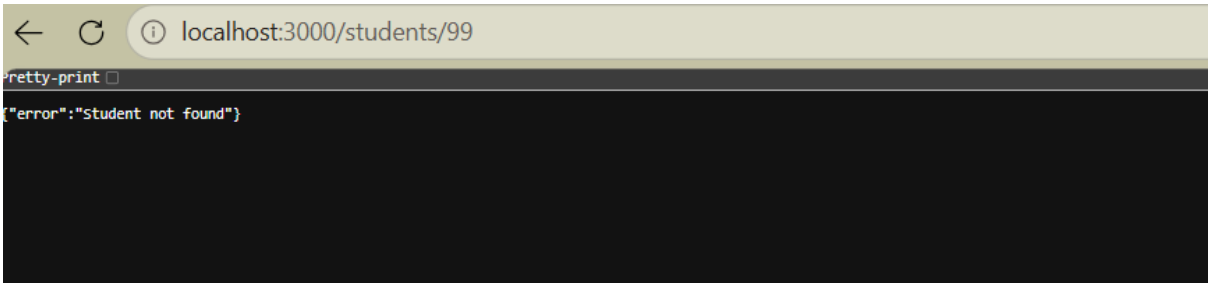
```
const http = require("http");
const students = [
    { id: 1, name: "Vikrant", course: "CSE" },
    { id: 2, name: "Rahul", course: "CSE" },
    { id: 3, name: "Aman", course: "CSE" }
];
const server = http.createServer((req, res) => {
    if (req.method === "GET" && req.url === "/") {
        res.writeHead(200, { "Content-Type": "text/html" });
        res.end("<h1>Welcome to Student Server</h1>");
    }
    else if (req.method === "GET" && req.url === "/students") {
        res.writeHead(200, { "Content-Type": "application/json" });
        res.end(JSON.stringify(students));
    }
    else if (req.method === "GET" && req.url.startsWith("/students/")) {
        const id = parseInt(req.url.split("/")[2]);
        const student = students.find((s) => s.id === id);
        if (student) {
            res.writeHead(200, { "Content-Type": "application/json" });
            res.end(JSON.stringify(student));
        } else {
            res.writeHead(404, { "Content-Type": "application/json" });
            res.end(JSON.stringify({ error: "Student not found" }));
        }
    }
    else {
        res.writeHead(404, { "Content-Type": "text/html" });
        res.end("<h1>404 - Page Not Found</h1>");
    }
});
server.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

Output ->









Task 8: Query String Parsing

Extend server.js (or create query-demo.js) with a route /search that reads a query parameter (e.g. /search?keyword=node) using the url module and echoes it back in the response.

Use: require('url').parse(req.url, true).query

Solution -> server1.js

Lab Sheet 4 > JS server1.js > ...

```
const url = require("url");
const http = require("http");
const students = [
    { id: 1, name: "Vikrant", course: "CSE" },
    { id: 2, name: "Rahul", course: "CSE" },
    { id: 3, name: "Aman", course: "CSE" }
];
const server = http.createServer((req, res) => {
    if (req.method === "GET" && req.url === "/") {
        res.writeHead(200, { "Content-Type": "text/html" });
        res.end("<h1>Welcome to Student Server</h1>");
    }
    else if (req.method === "GET" && req.url === "/students") {
        res.writeHead(200, { "Content-Type": "application/json" });
        res.end(JSON.stringify(students));
    }
    else if (req.method === "GET" && req.url.startsWith("/students/")) {
        const id = parseInt(req.url.split("/")[2]);
        const student = students.find((s) => s.id === id);
        if (student) {
            res.writeHead(200, { "Content-Type": "application/json" });
            res.end(JSON.stringify(student));
        } else {
            res.writeHead(404, { "Content-Type": "application/json" });
            res.end(JSON.stringify({ error: "Student not found" }));
        }
    }
    else if (req.method === "GET" && req.url.startsWith("/search")) {
        const parsedUrl = url.parse(req.url, true);
        const keyword = parsedUrl.query.keyword;
        res.writeHead(200, { "Content-Type": "text/html" });
        res.end(`<h1>Search Keyword: ${keyword}</h1>`);
    }
    else {
        res.writeHead(404, { "Content-Type": "text/html" });
        res.end("<h1>404 - Page Not Found</h1>");
    }
});
server.listen(3000, () => {
    console.log("Server running at http://localhost:3000");
});
```

Output



Task 9: Reading a Large File with Streams

Write stream-demo.js that creates a text file with at least 50 lines using a loop, then reads it back using fs.createReadStream and logs each chunk size received.

File: stream-demo.js Use: fs.createReadStream, 'data' event, 'end' event

Solution -> stream-demo.js

```
Lab Sheet 4 > JS stream-demo.js > ...
1   const fs = require("fs");
2   let content = "";
3   for (let i = 1; i <= 50; i++) {
4       content += `This is line ${i}\n`;
5   }
6   fs.writeFileSync("large-file.txt", content);
7   console.log("File created with 50 lines.");
8   const readStream = fs.createReadStream("large-file.txt");
9   readStream.on("data", (chunk) => {
10      console.log("Chunk received:", chunk.length, "bytes");
11  });
12  readStream.on("end", () => {
13      console.log("Finished reading the file.");
14  });
```

Output ->

```
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4> node .\stream-demo.js
File created with 50 lines.
Chunk received: 791 bytes
Finished reading the file.
```

Task 10: npm and an External Package

Run npm init -y, install nanoid@3, then write id-gen.js that generates and prints 5 unique IDs using nanoid.

Run: npm init -y Run: npm install nanoid@3 File: id-gen.js

Solution ->

```
Lab Sheet 4 > JS id-gen.js > ...
1   const { nanoid } = require("nanoid");
2   for (let i = 1; i <= 5; i++) {
3       console.log(`ID ${i}: ${nanoid()}`);
4   }
5
```

Output ->

```
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4> node .\id-gen.js
ID 1: WnfVDOSVPO13VOuep6OiP
ID 2: l9adqMa6rWqq9qzQ0C5a-
ID 3: SwstZdioc9A7688HKcNFW
ID 4: 4frY19jUi7R2W5j7JrX_S
ID 5: F5b59yjw5RS9A81rfywVx
```

Task 11: Command-Line Arguments

Write calc-cli.js that accepts two numbers and an operator as command-line arguments (e.g. node calc-cli.js 10 5 +) and prints the result. Support +, -, *, /.

File: calc-cli.js Use: process.argv Run: node calc-cli.js 10 5 +

Solution -> calc-cli.js

```
Lab Sheet 4 > JS calc-cli.js > ...
const num1 = parseFloat(process.argv[2]);
const num2 = parseFloat(process.argv[3]);
const operator = process.argv[4];
let result;
switch (operator) {
    case "+":
        result = num1 + num2;
        break;
    case "-":
        result = num1 - num2;
        break;
    case "*":
        result = num1 * num2;
        break;
    case "/":
        if (num2 === 0) {
            console.log("Error: Cannot divide by zero.");
            process.exit();
        }
        result = num1 / num2;
        break;
    default:
        console.log("Invalid operator. Use +, -, * or /.");
        process.exit();
}
console.log("Result:", result);
```

Output ->

```
PS C:\Users\DELL\OneDrive\Desktop\c++ Programs\.vscode\Lab Sheet 4> node calc-cli.js 10 5 +
Result: 15
```

Task 12: Environment Variables

Write env-demo.js that reads a PORT value from process.env (default to 4000 if not set) and starts an HTTP server on that port, logging the active port on start.

File: env-demo.js Run (Linux/Mac): PORT=5000 node env-demo.js Run (Windows): set PORT=5000 && node env-demo.js

Solution -> env-demo.js

```
Lab Sheet 4 > JS env-demo.js > ...
const http = require("http");
const PORT = process.env.PORT || 4000;
const server = http.createServer((req, res) => {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end(`<h1>Server is running on port ${PORT}</h1>`);
});
server.listen(PORT, () => {
    console.log(`Server started on port ${PORT}`);
});
```

Output ->

